

Tabla de contenidos

Este manual describe paso a paso cómo configurar las tres piezas clave de la plataforma ChatFlow: pruebas E2E, documentación API y facturación real con Stripe. Cada capítulo es autónomo: puedes implementarlos en paralelo o en el orden que prefieras.

Capítulo 1 · Visión general	3
1.1 · Arquitectura de las tres piezas	3
1.2 · Requisitos previos	3
1.3 · Convenciones del manual	3
Capítulo 2 · Pruebas E2E con Playwright	5
2.1 · Instalación y configuración inicial	5
2.2 · Estructura de los tests	6
2.3 · Ejemplo: test de endpoint público	6
2.4 · Tests con autenticación	7
2.5 · Tests de UI con captura de errores	7
2.6 · Scripts de npm y ejecución	8
2.7 · Lista de verificación	8
Capítulo 3 · Documentación API con OpenAPI	9
3.1 · Instalación de dependencias	9
3.2 · Estructura de la spec OpenAPI	9
3.3 · Endpoint que sirve la spec	10
3.4 · Página /docs con Scalar UI	11
3.5 · Convenciones y buenas prácticas	12
3.6 · Test E2E para la documentación	12
3.7 · Lista de verificación	12
Capítulo 4 · Integración Stripe real	13
4.1 · Modelo de datos en Prisma	13
4.2 · Instalación y cliente singleton	13
4.3 · Configurar productos y precios en Stripe	14
4.4 · Endpoint de checkout	15
4.5 · Webhook con verificación de firma	16
4.6 · Webhook local con Stripe CLI	17
4.7 · Customer Portal	17
4.8 · Endpoint público de planes	18

4.9 · Variables de entorno	18
4.10 · Lista de verificación	18
Capítulo 5 · Verificación y operación	19
5.1 · Smoke test completo	19
5.2 · Tests E2E en CI	19
5.3 · Debug de webhooks que no llegan	20
5.4 · Monitoreo continuo	20
Apéndice A · Variables de entorno	21
Apéndice B · Solución de problemas	22
B.1 · Playwright	22
B.2 · OpenAPI / Scalar	22
B.3 · Stripe	22
B.4 · Comandos útiles	23

Capítulo 1 · Visión general

ChatFlow es una plataforma SaaS multi-canal para construir chatbots con constructor visual de flujos. Antes de llevarla a producción en cualquier negocio, hay tres piezas no negociables que toda plataforma SaaS moderna debe tener: **pruebas automatizadas** que validen que los flujos críticos siguen funcionando tras cada cambio, **documentación API auto-actualizable** que los integradores puedan consumir sin pedirte ayuda, y **facturación real** que cobre a tus clientes de forma confiable y cumpla con las normativas fiscales del país donde operas.

Este manual detalla cómo configurar esas tres piezas en un proyecto nuevo o existente basado en Next.js 16 + Prisma + PostgreSQL. Cada capítulo incluye: archivos a crear, comandos a ejecutar, variables de entorno necesarias, y una lista de verificación final. La configuración de SUNAT (facturación electrónica de Perú) se deja pendiente para un documento posterior: Stripe funciona perfectamente sin SUNAT y emite facturas propias que la mayoría de negocios puede usar directamente.

1.1 · Arquitectura de las tres piezas

Las tres piezas son ortogonales pero comparten el mismo patrón: usan librerías estándar de la industria, se configuran vía variables de entorno, y exponen endpoints HTTP que se pueden probar de forma aislada. La tabla siguiente resume el rol de cada una y los componentes principales.

Pieza	Rol	Stack	Carpeta principal
E2E	Validar flujos críticos tras cada deploy	Playwright + Chromium	tests/e2e/
OpenAPI	Documentación API auto-actualizable	openapi-types + Scalar UI	src/lib/openapi-spec.ts + /docs
Stripe	Cobros recurrentes y facturación	stripe-node + webhook + portal	src/app/api/stripe/ + src/lib/stripe.ts

1.2 · Requisitos previos

Antes de empezar necesitas un proyecto Next.js 16+ con TypeScript, una base de datos PostgreSQL (recomendamos Neon o Supabase para producción, o Postgres local para desarrollo), y Node.js 18+. Para la parte de Stripe necesitarás además una cuenta de Stripe (puede ser test mode mientras desarrollas). No se requiere experiencia previa con ninguna de las librerías: el manual cubre todo desde cero.

- **Node.js 18+** con soporte para TypeScript 5 y Next.js 16 (App Router).
- **PostgreSQL 14+** accesible vía Prisma. Para producción recomendamos Neon por su tier free generoso y branching para staging.
- **Cuenta Stripe** con acceso al Dashboard (test mode es suficiente para desarrollo; necesitarás live mode cuando cobres clientes reales).
- **Editor con soporte TypeScript** para autocompletar los tipos de Stripe y Playwright.
- **Git** para versionar los tests y la spec OpenAPI junto al código.

1.3 · Convenciones del manual

Los bloques de código están en monoespaciado con fondo gris claro. Los comandos que se ejecutan en terminal llevan prefijo "\$ ". Las rutas de archivos son relativas a la raíz del proyecto a menos que se indique lo contrario. Las notas y advertencias aparecen en recuadros amarillos. Cuando una variable de entorno lleva el prefijo `STRIPE_` o `SUNAT_`, es sensible y no debe commitearse al repositorio.

Nota: Si estás configurando estas piezas en un proyecto existente, te recomendamos crear una rama `feat/billing-tests-docs` y hacer commits por capítulo. Así puedes revertir un capítulo sin afectar los otros si algo sale mal.

Capítulo 2 · Pruebas E2E con Playwright

Las pruebas end-to-end (E2E) simulan un usuario real navegando tu aplicación: abren un navegador, hacen clic en botones, rellenan formularios y verifican que la UI responde correctamente. Son la última línea de defensa antes de production: si un test E2E pasa, tienes alta confianza de que el flujo completo (frontend + API + base de datos) funciona. Playwright es la librería moderna recomendada: soporta Chromium, Firefox y WebKit, tiene auto-waiting inteligente, y graba traces completos cuando un test falla para que puedas depurarlo offline.

2.1 · Instalación y configuración inicial

Playwright se distribuye como un paquete npm con sus propios binarios de navegador. La instalación es en dos pasos: primero el paquete npm y luego los binarios de navegador. Los binarios ocupan ~300MB pero son necesarios para ejecutar las pruebas en headless mode.

```
# Instalar Playwright como dependencia de desarrollo
npm install --save-dev @playwright/test

# Descargar los binarios de navegador (Chromium por defecto)
npx playwright install chromium

# Verificar la instalación
npx playwright --version
```

A continuación crea el archivo `playwright.config.ts` en la raíz del proyecto. Esta configuración define dónde están los tests, qué navegador usar, y cómo arrancar el dev server automáticamente cuando corres los tests en local. En CI se asume que el servidor ya está corriendo (lo arranca el propio pipeline).

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: './tests/e2e',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 1 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: process.env.CI
    ? [['github'], ['html', { open: 'never' }]]
    : 'list',
  timeout: 30_000,
  expect: { timeout: 5_000 },
  use: {
    baseURL: process.env.BASE_URL || 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  ],
  webServer: process.env.CI ? undefined : {
    command: 'npm run dev',
    url: 'http://localhost:3000/api/healthz',
    reuseExistingServer: true,
    timeout: 60_000,
  },
})
```

```
},
})
```

El campo `webServer` es clave: le dice a Playwright que arranque tu dev server antes de correr los tests y que espere a que `/api/healthz` responda 200. Si ya hay un servidor corriendo en el puerto 3000, lo reutiliza. En CI desactivamos este comportamiento porque el servidor lo arranca el pipeline separadamente.

2.2 · Estructura de los tests

Organiza los tests en la carpeta `tests/e2e/` con un archivo `.spec.ts` por dominio funcional. Cada archivo contiene un `test.describe` por recurso y un `test` por escenario. La granularidad recomendada es: **un test = un comportamiento verificable**. Si un test falla, debes poder entender qué funcionalidad se rompió mirando solo su nombre.

Archivo	Cobertura	Auth requerida
<code>public.spec.ts</code>	Endpoints públicos + aplicación de auth	No
<code>api-core.spec.ts</code>	Bots, Conversations, Contacts (CRUD completo)	Sí
<code>stripe.spec.ts</code>	Planes, checkout, portal, webhook signature	Mixto
<code>ui-smoke.spec.ts</code>	Home page, docs page, errores JS fatales	No

2.3 · Ejemplo: test de endpoint público

Los tests de endpoints públicos son los más sencillos: no requieren autenticación y validan que la API responde con el status y shape esperados. Son excelentes como smoke test en CI porque detectan regresiones graves en segundos.

```
// tests/e2e/public.spec.ts
import { test, expect } from '@playwright/test'

test.describe('Public endpoints', () => {
  test('GET /api/healthz returns 200', async ({ request }) => {
    const res = await request.get('/api/healthz')
    expect(res.status()).toBe(200)
    const body = await res.json()
    expect(body.status).toBe('ok')
  })

  test('GET /api/docs/json returns OpenAPI 3.1 spec', async ({ request }) => {
    const res = await request.get('/api/docs/json')
    expect(res.status()).toBe(200)
    const spec = await res.json()
    expect(spec.openapi).toBe('3.1.0')
    expect(spec.paths['/api/bots']).toBeDefined()
  })

  test('GET /api/stripe/plans returns catalog (public)', async ({ request }) => {
    const res = await request.get('/api/stripe/plans')
    expect(res.status()).toBe(200)
    const body = await res.json()
    expect(body.success).toBe(true)
    expect(body.data.length).toBeGreaterThanOrEqual(3)
  })
})
```

```
}}
```

2.4 · Tests con autenticación

Para endpoints protegidos necesitas pasar tu API key en el header `Authorization: Bearer cf_XXX`. La práctica recomendada es leer la key de una variable de entorno `ADMIN_API_KEY` y hacer `test.skip` si no está definida. Así el test pasa en CI (donde sí hay key) y se omite en local si no la configuraste.

```
// tests/e2e/api-core.spec.ts
import { test, expect } from '@playwright/test'

const API_KEY = process.env.ADMIN_API_KEY || ''
const headers = { Authorization: `Bearer ${API_KEY}` }

test.describe('Bots API', () => {
  test.skip(!API_KEY, 'ADMIN_API_KEY env var required')

  test('list bots returns 200', async ({ request }) => {
    const res = await request.get('/api/bots', { headers })
    expect(res.status()).toBe(200)
    const body = await res.json()
    expect(body.success).toBe(true)
    expect(Array.isArray(body.data)).toBe(true)
  })

  test('create + delete a bot', async ({ request }) => {
    const create = await request.post('/api/bots', {
      headers: { ...headers, 'Content-Type': 'application/json' },
      data: { name: 'E2E Test Bot ' + Date.now(), channels: 'whatsapp' },
    })
    expect([200, 201]).toContain(create.status())
    const created = await create.json()
    const botId = created.data?.id || created.id

    const del = await request.delete(`/api/bots/${botId}`, { headers })
    expect([200, 204]).toContain(del.status())
  })
})
```

2.5 · Tests de UI con captura de errores

Para tests de UI, lo más valioso no es solo verificar que el texto esperado está en pantalla, sino capturar errores JavaScript fatales que pueden ocurrir durante el render. El patrón siguiente captura todos los `pageerror` y falla el test si hay alguno que no sea de hidratación (que son tolerados por React 19).

```
// tests/e2e/ui-smoke.spec.ts
test('home page loads without fatal errors', async ({ page }) => {
  const errors: string[] = []
  page.on('pageerror', (err) => errors.push(err.message))

  const res = await page.goto('/')
  expect(res?.status()).toBe(200)
  const title = await page.title()
  expect(title).toContain('ChatFlow')

  // Toleramos errores de hidratación de React
  const fatalErrors = errors.filter(
    (e) => !e.includes('Hydration') && !e.includes('hydrat')
  )
})
```

```
)  
expect(fatalErrors, JSON.stringify(fatalErrors)).toEqual([])  
})
```

2.6 · Scripts de npm y ejecución

Agrega estos scripts a tu `package.json` para estandarizar cómo se ejecutan los tests. El modo `--ui` abre un inspector visual donde puedes ver cada paso del test, pausarlo y modificar selectores en vivo.

```
{  
  "scripts": {  
    "test:e2e": "playwright test",  
    "test:e2e:ui": "playwright test --ui",  
    "test:e2e:report": "playwright show-report"  
  }  
}
```

Para ejecutar los tests localmente necesitas el servidor corriendo. Playwright lo arranca automáticamente si no detecta ninguno en el puerto 3000. Para forzar un servidor concreto (por ejemplo, un deploy de preview en Vercel), pasa `BASE_URL` como variable de entorno.

```
# Local con dev server auto-arrancado  
npm run test:e2e  
  
# Modo interactivo (recomendado para escribir tests nuevos)  
npm run test:e2e:ui  
  
# Contra un deploy de preview  
BASE_URL=https://chatflow-preview.vercel.app \  
ADMIN_API_KEY=cf_xxx \  
npm run test:e2e  
  
# Ver reporte HTML de la última corrida  
npm run test:e2e:report
```

Nota: En CI (GitHub Actions, Vercel Preview Deployments) define siempre `CI=true` para activar los reintentos y reducir falsos negativos por flakes de red.

2.7 · Lista de verificación

- **@playwright/test** instalado como devDependency
- Binarios de Chromium descargados con `npx playwright install chromium`
- `playwright.config.ts` en la raíz con `webServer` configurado
- Al menos un `.spec.ts` por dominio en `tests/e2e/`
- Scripts `test:e2e`, `test:e2e:ui` y `test:e2e:report` en `package.json`
- Tests de endpoints públicos pasan sin configuración adicional
- Tests de endpoints protegidos se skippean correctamente sin `ADMIN_API_KEY`

Capítulo 3 · Documentación API con OpenAPI

OpenAPI 3.1 es el estándar de la industria para describir APIs REST. Una vez que tienes una spec OpenAPI válida, puedes generar automáticamente documentación interactiva (con botón "try-it-now"), clientes SDK en cualquier lenguaje, mocks para tests, y colecciones de Postman. La alternativa — mantener un README con ejemplos sueltos — siempre se desactualiza y genera fricción con los integradores. En este capítulo configuramos una spec OpenAPI 3.1 servida desde tu propia API y renderizada con Scalar (la UI moderna preferida sobre Swagger UI clásico).

3.1 · Instalación de dependencias

Necesitas tres paquetes: **openapi-types** para los tipos TypeScript de la spec, **@scalar/api-reference-react** para la UI interactiva, y opcionalmente **swagger-ui-react** si prefieres la UI clásica. Scalar es más moderna, más rápida y con mejor UX móvil.

```
# Dependencias de runtime
npm install @scalar/api-reference-react @scalar/api-reference

# Dependencias de tipos (devDependency)
npm install --save-dev openapi-types
```

3.2 · Estructura de la spec OpenAPI

La spec vive en `src/lib/openapi-spec.ts` como un objeto TypeScript tipado. Esto te da autocompletado en el editor y validación de tipos en build time. La estructura sigue la especificación OpenAPI 3.1: `info` (metadatos), `servers` (URLs base), `tags` (agrupaciones de endpoints), `components` (esquemas reutilizables), y `paths` (los endpoints propiamente dichos).

```
// src/lib/openapi-spec.ts
import type { OpenAPIV3_1 } from 'openapi-types'

export const openApiSpec: OpenAPIV3_1.Document = {
  openapi: '3.1.0',
  info: {
    title: 'ChatFlow API',
    version: '1.0.0',
    description: 'API REST de ChatFlow — plataforma omnicanal de chatbots.'
  },
  ## Autenticación
  Todas las peticiones requieren API Key:
  \`\`\`http
  Authorization: Bearer cf_XXXXXXXXXXXXXXXXX
  \`\`\`,
  contact: { name: 'Soporte', email: 'soporte@chatflow.pe' },
  license: { name: 'MIT', url: 'https://opensource.org/license/mit' },
  },
  servers: [
    { url: '/', description: 'Relative (mismo dominio)' },
    { url: 'https://api.chatflow.pe', description: 'Producción' },
  ],
  tags: [
    { name: 'Auth', description: 'API keys y permisos' },
    { name: 'Bots', description: 'Gestión de bots' },
  ],
```

```

{ name: 'Stripe', description: 'Facturación' },
],
components: {
  securitySchemes: {
    ApiKeyAuth: {
      type: 'apiKey',
      in: 'header',
      name: 'Authorization',
      description: 'Usa `Authorization: Bearer cf_xxx`',
    },
  },
  schemas: {
    Bot: {
      type: 'object',
      properties: {
        id: { type: 'string' },
        name: { type: 'string' },
        channels: { type: 'array', items: { type: 'string' } },
        status: { type: 'string', enum: ['active', 'paused', 'draft'] },
      },
      required: ['id', 'name'],
    },
  },
  paths: {
    '/api/bots': {
      get: {
        tags: ['Bots'],
        summary: 'Lista todos los bots',
        security: [{ ApiKeyAuth: [] }],
        responses: {
          '200': {
            description: 'Lista de bots',
            content: {
              'application/json': {
                schema: {
                  type: 'object',
                  properties: {
                    success: { type: 'boolean' },
                    data: { type: 'array', items: { $ref: '#/components/schemas/Bot' } },
                  },
                },
              },
            },
          },
          '401': { description: 'API key inválida o ausente' },
        },
      },
    },
  },
}

```

Nota: Mantén la spec en TypeScript (no en YAML). Te da autocompletado, validación de tipos y refactor seguro. Los YAMLS son propensos a errores de indentación que se descubren solo en runtime.

3.3 · Endpoint que sirve la spec

Crea un endpoint `/api/docs/json` que devuelva la spec como JSON. Es público (sin auth) para que herramientas externas como Postman, Insomnia o Stoplight puedan importarlo.

```
// src/app/api/docs/json/route.ts
import { NextResponse } from 'next/server'
import { openApiSpec } from '@lib/openapi-spec'

export const runtime = 'nodejs'

export async function GET() {
  return NextResponse.json(openApiSpec, {
    headers: { 'Cache-Control': 'no-store' },
  })
}
```

3.4 · Página /docs con Scalar UI

La UI de documentación vive en /docs y usa el componente `ApiReferenceReact` de Scalar. Es importante cargarlo con `dynamic` y `ssr: false` porque Scalar solo funciona en cliente. La página hace fetch de la spec JSON al montarse y se la pasa al componente.

```
// src/app/docs/page.tsx
'use client'
import dynamic from 'next/dynamic'
import { useEffect, useState } from 'react'

const ApiReferenceReact = dynamic(
  () => import('@scalar/api-reference-react').then((m) => m.ApiReferenceReact),
  { ssr: false, loading: () => <div>Cargando documentación...</div> }
)

export default function DocsPage() {
  const [spec, setSpec] = useState<object | null>(null)

  useEffect(() => {
    fetch('/api/docs/json')
      .then((r) => r.json())
      .then(setSpec)
      .catch(() => setSpec(null))
  }, [])

  if (!spec) return <div>Cargando...</div>

  return (
    <div style={{ height: '100vh', width: '100%' }}>
      <ApiReferenceReact
        configuration={{
          spec: { content: spec as any },
          theme: 'purple',
          layout: 'modern',
          darkMode: true,
          showSidebar: true,
          hideTestRequestButton: false,
          metaData: { title: 'ChatFlow API Docs' },
        }}
      />
    </div>
  )
}
```

Con esto, al abrir /docs en tu navegador ves la documentación completa con buscador, panel lateral, esquemas expandibles, ejemplos, y botón "Try it" que dispara requests reales contra tu API. No necesitas mantener un portal de docs aparte: la UI se actualiza sola cada vez que cambia la spec.

3.5 · Convenciones y buenas prácticas

- **Un endpoint = una entrada en paths.** Si añades un endpoint nuevo, añádelo también a la spec en el mismo PR. Es la única forma de mantener la doc sincronizada.
- **Usa \$ref para esquemas reutilizables.** El modelo Bot aparece en muchos endpoints: defínelo una vez en components/schemas y refléjalo con \$ref: '#/components/schemas/Bot'.
- **Documenta los errores.** Cada endpoint debe documentar al menos 400 (bad request), 401 (no auth) y 500 (server error) con ejemplos.
- **Descripciones en español si tu audiencia es hispana.** El título y la descripción de la spec son lo primero que ve el integrador.
- **Versionado semántico en info.version.** Bumpéa el minor cuando añades endpoints, el mayor cuando rompes compatibilidad.

3.6 · Test E2E para la documentación

La spec OpenAPI es un contrato: si rompes un endpoint sin actualizar la spec, los integradores se enteran cuando ya es tarde. El test E2E siguiente valida que la spec sigue sirviéndose correctamente y que los paths clave siguen presentes.

```
test('GET /api/docs/json returns OpenAPI 3.1 spec', async ({ request }) => {
  const res = await request.get('/api/docs/json')
  expect(res.status()).toBe(200)
  const spec = await res.json()
  expect(spec.openapi).toBe('3.1.0')
  expect(spec.info.title).toBe('ChatFlow API')
  expect(spec.paths['/api/bots']).toBeDefined()
  expect(spec.paths['/api/stripe/checkout']).toBeDefined()
})

test('GET /docs renders Scalar API reference', async ({ page }) => {
  await page.goto('/docs')
  await page.waitForLoadState('networkidle')
  const bodyText = await page.locator('body').innerText({ timeout: 10_000 })
  expect(bodyText.toLowerCase()).toContain('chatflow')
})
```

3.7 · Lista de verificación

- **openapi-types** y **@scalar/api-reference-react** instalados
- `src/lib/openapi-spec.ts` exportando un objeto tipado `OpenAPIV3_1.Document`
- Endpoint GET `/api/docs/json` devolviendo la spec como JSON
- Página `/docs` con Scalar cargando la spec dinámicamente
- Todos los endpoints públicos documentados con response 200 y 401 mínimo
- Test E2E que valida que `/api/docs/json` responde 200 con openapi 3.1.0

Capítulo 4 · Integración Stripe real

Stripe es el procesador de pagos estándar de la industria SaaS. Maneja compliance PCI-DSS, suscripciones recurrentes, dunning automático, y emite facturas que la mayoría de negocios puede usar directamente. En este capítulo configuramos el flujo completo: catálogo de planes, checkout, webhook con verificación de firma, customer portal, y la tabla `Subscription` local que mantiene el estado sincronizado. La integración funciona sin SUNAT: las facturas que emite Stripe son suficientes para la mayoría de casos. Cuando necesites facturación electrónica Perú (SUNAT), se añade encima sin romper nada.

4.1 · Modelo de datos en Prisma

Necesitas dos modelos en tu schema Prisma: **Subscription** (estado actual de la suscripción del cliente: plan, límites, fechas) e **Invoice** (registro de cada factura emitida, con su PDF y estado). Ambos se actualizan automáticamente desde el webhook de Stripe, no desde la UI del cliente.

```
// prisma/schema.prisma
model Subscription {
  id String @id @default(cuid())
  customerId String @unique // tu identificador interno
  plan String @default("free")
  status String @default("active")
  seats Int @default(1)
  conversationsLimit Int @default(100)
  messagesLimit Int @default(1000)
  stripeCustomerId String? @unique
  stripeSubscriptionId String? @unique
  stripePriceId String?
  cancelAtPeriodEnd Boolean @default(false)
  currentPeriodStart DateTime?
  currentPeriodEnd DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  invoices Invoice[]
}

model Invoice {
  id String @id @default(cuid())
  subscriptionId String?
  subscription Subscription? @relation(fields: [subscriptionId], references: [id])
  stripeInvoiceId String @unique
  amount Int // en centavos
  currency String @default("usd")
  status String @default("open") // open|paid|uncollectible|void
  pdfUrl String?
  hostedInvoiceUrl String?
  createdAt DateTime @default(now())
}
```

Después de editar el schema, ejecuta `npx prisma db push` para aplicar los cambios a la base de datos y `npx prisma generate` para regenerar el cliente TypeScript.

4.2 · Instalación y cliente singleton

```
# Instalar el SDK oficial de Stripe para Node
npm install stripe
```

El cliente Stripe debe ser un singleton para evitar crear conexiones innecesarias en serverless. Importante: **devuelve null si no hay STRIPE_SECRET_KEY configurada**, para que la app siga funcionando en entornos sin billing (dev local, PR previews, etc.).

```
// src/lib/stripe.ts
import Stripe from 'stripe'

let _stripe: Stripe | null = null

export function getStripe(): Stripe | null {
  if (_stripe) return _stripe
  const secretKey = process.env.STRIPE_SECRET_KEY
  if (!secretKey) return null
  _stripe = new Stripe(secretKey, {
    apiVersion: '2024-12-18.acacia' as any,
    typescript: true,
  })
  return _stripe
}

// Catálogo de planes (debe coincidir con los products/prices en Stripe Dashboard)
export const PLANS = [
  { id: 'free', name: 'Free', priceMonthly: 0, conversationsLimit: 100, seats: 1 },
  { id: 'pro', name: 'Pro', priceMonthly: 4900, conversationsLimit: 5000, seats: 3,
    stripePriceId: process.env.STRIPE_PRICE_PRO_MONTHLY,
    stripePriceIdAnnual: process.env.STRIPE_PRICE_PRO_ANNUAL },
  { id: 'business', name: 'Business', priceMonthly: 14900, conversationsLimit: 25000,
    seats: 10,
    stripePriceId: process.env.STRIPE_PRICE_BUSINESS_MONTHLY,
    stripePriceIdAnnual: process.env.STRIPE_PRICE_BUSINESS_ANNUAL },
] as const

export const STRIPE_WEBHOOK_SECRET = process.env.STRIPE_WEBHOOK_SECRET || ''

export function constructStripeEvent(payload: string | Buffer, signature: string):
Stripe.Event {
  const stripe = getStripe()
  if (!stripe) throw new Error('Stripe no configurado')
  return stripe.webhooks.constructEvent(payload, signature, STRIPE_WEBHOOK_SECRET)
}
```

4.3 · Configurar productos y precios en Stripe

Tienes dos opciones para crear los productos y precios en Stripe: manualmente vía Dashboard, o automáticamente con el script `scripts/setup-stripe.ts` que viene con el proyecto. El script es idempotente: lo puedes correr varias veces y solo crea lo que falta. Recomendamos el script porque evita errores manuales y deja el setup reproducible.

```
# Opción A: Script asistido (recomendado)
# Crea products + prices automáticamente y escribe los IDs en .env
STRIPE_SECRET_KEY=sk_test_xxx npx tsx scripts/setup-stripe.ts --write

# Opción B: Manual vía Dashboard
# 1. Ve a https://dashboard.stripe.com/products
# 2. Crea producto "ChatFlow Pro" con price mensual $49 USD y anual $470 USD
# 3. Crea producto "ChatFlow Business" con price mensual $149 USD y anual $1430 USD
# 4. Copia los price_xxx a tu .env
```

El output del script te muestra los cuatro `price_xxx` que debes copiar a tu `.env`:
`STRIPE_PRICE_PRO_MONTHLY`, `STRIPE_PRICE_PRO_ANNUAL`,
`STRIPE_PRICE_BUSINESS_MONTHLY` y `STRIPE_PRICE_BUSINESS_ANNUAL`. Sin estos cuatro valores, el endpoint de checkout devuelve 500.

4.4 · Endpoint de checkout

El endpoint `POST /api/stripe/checkout` crea una Stripe Checkout Session y devuelve la URL a la que el frontend debe redirigir al usuario. Recibe el plan solicitado y tu `customerId` interno (puede ser el `user id`, `workspace id`, etc.). El `customerId` viaja como metadata y permite al webhook asociar el evento de vuelta a tu base de datos local.

```
// src/app/api/stripe/checkout/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { getStripe, PLANS } from '@lib/stripe'
import { db } from '@lib/db'
import { requireAuth } from '@lib/auth'

export const runtime = 'nodejs'

export async function POST(req: NextRequest) {
  const auth = await requireAuth(req)
  if (!auth.success) return auth.response

  const stripe = getStripe()
  if (!stripe) return NextResponse.json({ error: 'Stripe no configurado' }, { status: 503 })

  const body = await req.json().catch(() => ({}))
  const { plan: planId, annual, customerId, customerEmail } = body
  const plan = PLANS.find((p) => p.id === planId)
  if (!plan || plan.id === 'free' || plan.id === 'enterprise') {
    return NextResponse.json({ error: 'Plan inválido' }, { status: 400 })
  }
  const priceId = annual ? plan.stripePriceIdAnnual : plan.stripePriceId
  if (!priceId) {
    return NextResponse.json({ error: 'Price ID no configurado' }, { status: 500 })
  }

  // Reusa el stripeCustomerId si el cliente ya tenía uno
  const existingSub = await db.subscription.findUnique({ where: { customerId } })
  const stripeCustomerId = existingSub?.stripeCustomerId || undefined

  const session = await stripe.checkout.sessions.create({
    mode: 'subscription',
    customer: stripeCustomerId,
    customer_email: stripeCustomerId ? undefined : customerEmail,
    line_items: [{ price: priceId, quantity: 1 }],
    success_url: `${process.env.NEXT_PUBLIC_APP_URL}/?billing=success&session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/?billing=cancel`,
    metadata: { planId: plan.id, customerId, annual: String(annual) },
    subscription_data: { metadata: { planId: plan.id, customerId } },
    allow_promotion_codes: true,
    billing_address_collection: 'auto',
    customer_creation: stripeCustomerId ? undefined : 'always',
  })

  return NextResponse.json({ url: session.url, sessionId: session.id })
}
```

```
}
```

4.5 · Webhook con verificación de firma

El webhook `POST /api/stripe/webhook` es la pieza más crítica: recibe eventos de Stripe (pago realizado, suscripción actualizada, etc.) y actualiza tu base de datos local. Tres cosas son obligatorias: (1) leer el body como texto plano (no JSON parseado, porque la firma se calcula sobre el raw), (2) verificar la firma con `stripe.webhooks.constructEvent`, y (3) responder 200 rápido para evitar reintentos de Stripe.

```
// src/app/api/stripe/webhook/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { db } from '@lib/db'
import { getStripe, constructStripeEvent, STRIPE_WEBHOOK_SECRET, getPlan } from
 '@lib/stripe'

export const runtime = 'nodejs'
export const dynamic = 'force-dynamic'

export async function POST(req: NextRequest) {
  const stripe = getStripe()
  if (!stripe) return NextResponse.json({ error: 'Stripe no configurado' }, { status: 503 })
  if (!STRIPE_WEBHOOK_SECRET) return NextResponse.json({ error: 'Sin webhook secret' }, {
    status: 503 })

  const signature = req.headers.get('stripe-signature')
  if (!signature) return NextResponse.json({ error: 'Falta signature' }, { status: 400 })

  const rawBody = await req.text() // ← CRÍTICO: texto plano, no JSON
  let event
  try {
    event = constructStripeEvent(rawBody, signature)
  } catch (e) {
    return NextResponse.json({ error: 'Firma inválida' }, { status: 400 })
  }

  try {
    switch (event.type) {
      case 'checkout.session.completed':
        await handleCheckoutCompleted(event.data.object as any)
        break
      case 'customer.subscription.updated':
        await handleSubscriptionUpdated(event.data.object as any)
        break
      case 'customer.subscription.deleted':
        await handleSubscriptionDeleted(event.data.object as any)
        break
      case 'invoice.payment_succeeded':
        await handleInvoicePaid(event.data.object as any)
        break
      // ... otros eventos
    }
    return NextResponse.json({ received: true, type: event.type })
  } catch (e) {
    console.error('Stripe webhook error', e)
    return NextResponse.json({ error: 'Error procesando webhook' }, { status: 500 })
  }
}
```

Atención: El body debe leerse con `await req.text()` y **nunca** con `await req.json()`. La firma HMAC de Stripe se calcula sobre el raw payload; si Next.js lo parsea como JSON y luego lo re-serializa, los bytes cambian y la firma nunca coincide.

4.6 · Webhook local con Stripe CLI

Para desarrollo local necesitas el Stripe CLI, que escucha eventos de tu cuenta Stripe (test mode) y los reenvía a tu localhost. El CLI te imprime un `whsec_xxx` que debes usar como `STRIPE_WEBHOOK_SECRET` en tu `.env`. Sin ese secret, la verificación de firma siempre falla.

```
# Instala Stripe CLI: https://stripe.com/docs/stripe-cli
# En una terminal, escucha y reenvía:
stripe listen --forward-to localhost:3000/api/stripe/webhook

# Output:
# > Ready! Your webhook signing secret is whsec_XXXXXXXX (^C to quit)
# Copia whsec_xxx a tu .env como STRIPE_WEBHOOK_SECRET

# En otra terminal, dispara eventos de prueba:
stripe trigger checkout.session.completed
stripe trigger invoice.payment_succeeded
stripe trigger customer.subscription.deleted
```

Hemos incluido un script `npm run stripe:listen` en el `package.json` que ejecuta el comando anterior automáticamente. Recuerda actualizar `STRIPE_WEBHOOK_SECRET` en tu `.env` cada vez que reinicies el CLI: el secret cambia entre sesiones.

4.7 · Customer Portal

El Customer Portal de Stripe es una página hospedada en `billing.stripe.com` donde el cliente puede cambiar método de pago, ver facturas anteriores, actualizar plan o cancelar la suscripción. Te ahorra construir toda esa UI. El endpoint `POST /api/stripe/portal` crea una sesión del portal y devuelve la URL a la que redirigir al usuario.

```
// src/app/api/stripe/portal/route.ts
export async function POST(req: NextRequest) {
  const auth = await requireAuth(req)
  if (!auth.success) return auth.response

  const stripe = getStripe()
  if (!stripe) return NextResponse.json({ error: 'Stripe no configurado' }, { status: 503 })

  const { customerId } = await req.json().catch(() => ({}))
  if (!customerId) return NextResponse.json({ error: 'customerId requerido' }, { status: 400 })

  const sub = await db.subscription.findUnique({ where: { customerId } })
  if (!sub?.stripeCustomerId) {
    return NextResponse.json({ error: 'Sin suscripción activa' }, { status: 404 })
  }

  const session = await stripe.billingPortal.sessions.create({
    customer: sub.stripeCustomerId,
    return_url: `${process.env.NEXT_PUBLIC_APP_URL}/?billing=portal`,
  })
  return NextResponse.json({ url: session.url })
}
```

4.8 · Endpoint público de planes

El endpoint `GET /api/stripe/plans` es público (sin auth) y devuelve el catálogo de planes con sus precios y límites. Lo usa la página de pricing pública para renderizar la tabla de comparación. Es importante que sea público: los clientes potenciales deben ver los precios sin registrarse.

4.9 · Variables de entorno

Todas las variables de Stripe empiezan con `STRIPE_`. Las que empiezan con `STRIPE_PRICE_` son los IDs de precio que copias del Dashboard (o que el script `setup-stripe.ts` escribe por ti). En producción usas `sk_live_`, en test `sk_test_`. Nunca mezcles: si usas live con price IDs de test, Stripe rechaza la transacción.

Variable	Ejemplo	Origen
<code>STRIPE_SECRET_KEY</code>	<code>sk_test_51NxXXX...</code>	Stripe Dashboard → Developers → API keys
<code>STRIPE_WEBHOOK_SECRET</code>	<code>whsec_xXXXX...</code>	Output del stripe listen (dev) o Dashboard → Webhooks (prod)
<code>STRIPE_PRICE_PRO_MONTHLY</code>	<code>price_1PxXX...</code>	Dashboard → Products o script <code>setup-stripe.ts</code>
<code>STRIPE_PRICE_PRO_ANNUAL</code>	<code>price_1PxXX...</code>	Dashboard → Products o script <code>setup-stripe.ts</code>
<code>STRIPE_PRICE_BUSINESS_MONTHLY</code>	<code>price_1PxXX...</code>	Dashboard → Products o script <code>setup-stripe.ts</code>
<code>STRIPE_PRICE_BUSINESS_ANNUAL</code>	<code>price_1PxXX...</code>	Dashboard → Products o script <code>setup-stripe.ts</code>
<code>NEXT_PUBLIC_APP_URL</code>	<code>https://app.chatflow.pe</code>	Tu dominio de producción

4.10 · Lista de verificación

- Paquete **stripe** instalado
- `src/lib/stripe.ts` con `getStripe()` singleton y catálogo de planes
- Modelos **Subscription** e **Invoice** en Prisma
- Endpoint `POST /api/stripe/checkout` con auth
- Endpoint `POST /api/stripe/webhook` que verifica firma con `req.text()`
- Endpoint `POST /api/stripe/portal` con auth
- Endpoint `GET /api/stripe/plans` público
- Variables de entorno `STRIPE_*` configuradas en dev y producción
- Stripe CLI escuchando en desarrollo local
- Webhook de producción creado en Stripe Dashboard → Webhooks
- Test E2E que valida que la firma inválida devuelve 400

Capítulo 5 · Verificación y operación

Una vez configuradas las tres piezas, este capítulo te muestra cómo verificar que todo funciona end-to-end y cómo operarlas en el día a día. Incluye comandos de smoke test, flujos de depuración para los problemas más comunes, y un runbook para cuando algo falle en producción. La regla de oro es: si los tests E2E pasan y el webhook de Stripe responde 200, tienes alta confianza de que el sistema funciona. Si algo falla, los pasos siguientes te dicen dónde mirar.

5.1 · Smoke test completo

Después de cualquier deploy, ejecuta este smoke test de 30 segundos. Valida que la API responde, que la documentación carga, que el catálogo de planes es accesible, y que los endpoints protegidos rechazan peticiones sin auth. Si todos los checks pasan, el deploy es sano.

```
# 1. Healthcheck
curl -s https://TU_DOMINIO/api/healthz | jq .
# Esperado: { "status": "ok", ... }

# 2. OpenAPI spec
curl -s https://TU_DOMINIO/api/docs/json | jq '.info.title'
# Esperado: "ChatFlow API"

# 3. Planes públicos
curl -s https://TU_DOMINIO/api/stripe/plans | jq '.data | length'
# Esperado: 3 o 4 (free, pro, business, [enterprise])

# 4. Auth enforcement (debe fallar con 401)
curl -s -o /dev/null -w "%{http_code}" https://TU_DOMINIO/api/bots
# Esperado: 401

# 5. Webhook signature inválida (debe fallar con 400)
curl -s -X POST https://TU_DOMINIO/api/stripe/webhook \
-H "stripe-signature: invalid" \
-H "Content-Type: application/json" \
-d '{"id":"evt_test"}'
# Esperado: {"error":"Firma inválida"}
```

5.2 · Tests E2E en CI

En CI (Vercel Preview Deployments, GitHub Actions) los tests E2E deben correr en cada PR. La variable `CI=true` activa reintentos y desactiva el auto-arranque del dev server. Pasa `BASE_URL` apuntando al deploy de preview y `ADMIN_API_KEY` con una key de test dedicada para CI.

```
# Ejemplo: GitHub Actions
name: E2E Tests
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test
```

```
env:  
CI: 'true'  
BASE_URL: ${ secrets.PREVIEW_URL }  
ADMIN_API_KEY: ${ secrets.CI_API_KEY }
```

5.3 · Debug de webhooks que no llegan

El problema más común con Stripe es que los webhooks no llegan o no se procesan correctamente. Sigue estos pasos en orden: (1) verifica que el endpoint devuelve 200 con curl, (2) revisa el log de eventos en Stripe Dashboard → Developers → Events, (3) si los eventos aparecen como "failed", mira el response body que Stripe guardó — suele contener el error exacto.

- **Webhook retorna 400 "Firma inválida":** el `STRIPE_WEBHOOK_SECRET` en tu `.env` no coincide con el del endpoint. En dev local el secret cambia cada vez que reinicias `stripe listen`.
- **Webhook retorna 503 "Stripe no configurado":** falta `STRIPE_SECRET_KEY` en las variables de entorno. Verifica que esté en el dashboard de Vercel y que el deploy haya terminado.
- **Eventos aparecen como sent pero no procesados:** tu endpoint está caído o respondiendo 500. Revisa los logs de Vercel en tiempo real mientras Stripe reintenta.
- **Subscription no se crea en la DB local:** el `customerId` del metadata de la Checkout Session no coincide con ningún registro en tu tabla Subscription. Verifica que lo estás pasando correctamente al crear la sesión.

5.4 · Monitoreo continuo

Para producción necesitas visibilidad sobre tres cosas: (1) que los tests E2E pasan en cada deploy, (2) que el webhook de Stripe responde 200 consistentemente, y (3) que las suscripciones activas crecen o se mantienen. Mínimo recomendado: un cron que ejecute los tests E2E cada hora contra producción, alertas de Vercel/Stripe para webhooks fallidos, y un dashboard simple que muestre MRR y suscripciones activas (lo puedes construir con una query SQL directa).

Apéndice A · Variables de entorno

Lista completa de variables de entorno que el proyecto usa. Las marcadas como **Requerida** deben estar siempre presentes; las **Opcional** permiten que la app arranque sin ellas pero deshabilitan la funcionalidad correspondiente. Las marcadas **Sensible** no deben commitearse al repositorio: van en el dashboard de Vercel (o equivalente) y en el `.env` local que está en `.gitignore`.

Variable	Categoría	Req	Sensible
DATABASE_URL	Base de datos	Sí	Sí
DIRECT_URL	Base de datos (migraciones)	Sí	Sí
GLM_API_KEY	IA (Z.ai)	No	Sí
GLM_API_URL	IA (Z.ai)	No	No
STRIPE_SECRET_KEY	Billing	Sí (para cobrar)	Sí
STRIPE_WEBHOOK_SECRET	Billing	Sí (para webhook)	Sí
STRIPE_PRICE_PRO_MONTHLY	Billing	Sí (plan pro)	No
STRIPE_PRICE_PRO_ANNUAL	Billing	No	No
STRIPE_PRICE_BUSINESS_MONTHLY	Billing	Sí (plan business)	No
STRIPE_PRICE_BUSINESS_ANNUAL	Billing	No	No
NEXT_PUBLIC_APP_URL	App URL	Sí (en prod)	No
ADMIN_API_KEY	E2E tests	No (solo CI)	Sí
BASE_URL	E2E tests	No (default localhost)	No
SUNAT_ENABLED	SUNAT	No (default off)	No
SUNAT_ISSUER_RUC	SUNAT	No	No
SUNAT_SENDER_URL	SUNAT	No	Sí
SUNAT_SENDER_TOKEN	SUNAT	No	Sí

Atención: El archivo `.env.example` del repositorio contiene todas estas variables comentadas con descripciones. Cópialo a `.env` y rellena los valores. Nunca commitees el `.env` real.

Apéndice B · Solución de problemas

Errores más comunes y cómo resolverlos. Si tu problema no está aquí, revisa los logs en Vercel (o tu plataforma de deploy), los eventos en Stripe Dashboard → Developers → Events, y los traces de Playwright que se guardan en `test-results/` tras cada falla.

B.1 · Playwright

Síntoma	Causa probable	Solución
Error: "Executable doesn't exist"	Navegadores no instalados	<code>npx playwright install chromium</code>
Tests pasan en local pero fallan en CI	Flaky timing o variable CI no seteada	Añade retries: <code>process.env.CI ? 1 : 0</code>
Test.skip salta todos los tests	ADMIN_API_KEY no está en el entorno	Exporta la variable antes de correr el test
Error "page.goto: net::ERR_CONNECTION_REFUSED"	Servidor no está corriendo	El webServer del config debería arrancarlo; revisa que <code>npm run dev</code> funciona
Timeout en <code>waitForLoadState(networkidle)</code>	Hay requests de fondo (analytics, sockets) que no terminan	Usa <code>waitForLoadState("domcontentloaded")</code> o un <code>waitForTimeout</code>

B.2 · OpenAPI / Scalar

Síntoma	Causa probable	Solución
/docs muestra "Cargando..." infinito	El fetch a /api/docs/json falla	Verifica que el endpoint responde 200 con curl
Scalar monta pero no ve endpoints	El objeto paths está vacío o mal formado	Imprime la spec con jq y valida que paths tiene entradas
Error de TypeScript en openApi-spec.ts	Versión incorrecta de openapi-types	<code>npm install openapi-types@12</code> — la 13 cambió la API
"Try it" falla con CORS	Backend en otro dominio sin CORS configurado	Configura headers CORS en next.config.ts o sirve todo desde el mismo dominio

B.3 · Stripe

Síntoma	Causa probable	Solución
Checkout devuelve 500 "Price ID no configurado"	Falta STRIPE_PRICE_*_MONTHLY en .env	Ejecuta <code>scripts/setup-stripe.ts --write</code>
Webhook devuelve 400 "Firma inválida"	STRIPE_WEBHOOK_SECRET incorrecto	En dev: copia el secret del stripe listen. En prod: copia del Dashboard → Webhooks

Síntoma	Causa probable	Solución
Webhook devuelve 503 "Stripe no configurado"	Falta STRIPE_SECRET_KEY	Añade sk_test_xxx (dev) o sk_live_xxx (prod) al .env
Subscription no se crea en la DB	customerId del metadata no coincide	Verifica que pasas customerId al crear la Checkout Session
Portal devuelve 404 "Sin suscripción activa"	El cliente no tiene stripeCustomerId en la DB	Espera a que llegue el evento checkout.session.completed
Facturas se duplican en la DB	Webhook se llamó dos veces (Stripe reintenta)	Usa upsert con stripeInvoiceId como key única (ya lo hace el código del manual)

B.4 · Comandos útiles

```
# Ver la spec OpenAPI formateada
curl -s https://TU_DOMINIO/api/docs/json | jq .

# Test manual del webhook con evento real de Stripe (dev local)
stripe trigger checkout.session.completed
stripe trigger invoice.payment_succeeded
stripe trigger customer.subscription.deleted

# Ver eventos enviados por Stripe en dev local
stripe events list

# Reenviar un evento específico
stripe events resend evt_12345

# Listar suscripciones activas en tu DB
psql $DATABASE_URL -c "SELECT id, plan, status, 'currentPeriodEnd' FROM 'Subscription'
WHERE status = \'active\';"

# Listar facturas pagadas del último mes
psql $DATABASE_URL -c "SELECT id, amount, currency, status FROM 'Invoice' WHERE status
= \'paid\' AND 'createdAt' > NOW() - INTERVAL \'30 days\';"

# Ejecutar un único test de Playwright
npx playwright test tests/e2e/stripe.spec.ts -g "webhook endpoint rejects"

# Ver el trace de un test fallido
npx playwright show-trace test-results/.../trace.zip
```

Para soporte o reportar errores en este manual, contacta al equipo de DevOps o abre un issue en el repositorio del proyecto.